

Production Search Typeahead System

Student: Aditya Kumar Rai

SST Email: aditya.24bcs10178@sst.scaler.com

Stack: Java 21, Spring Boot, PostgreSQL, OpenSearch, Redis, Kafka

1. Architecture Diagram And Explanation

This is the high-level system architecture, not a repository or package diagram. The design separates low-latency read traffic from durable write processing so typeahead requests stay fast while query counts and trending signals are updated reliably.

```
Users / Browser
|
v
CDN / Load Balancer / API Gateway
|
v
Spring Boot API Service Replicas
|
|-- GET /suggest?q=<prefix>&rank=<count|hybrid>
|   |
|   v
|   Suggestion Service
|   |
|   v
|   Consistent Hash Ring
|   |
|   v
|   Redis Prefix Cache Shards
|   |
|   | cache miss
|   v
|   OpenSearch Suggestion Index
|
|-- POST /search
|   |
|   v
|   Kafka search-events topic
|   |
|   v
|   Batch Writer Workers
```

```
|  
|-- PostgreSQL query counts and trend buckets  
|-- OpenSearch weight refresh  
|-- Redis affected-prefix invalidation
```

Observability: Spring Boot Actuator + Micrometer metrics

The default local profile uses in-memory implementations so the evaluator can run the project immediately after cloning. The Docker profile switches to PostgreSQL, OpenSearch, Redis shards, and Kafka.

2. Dataset Source And Loading Instructions

The selected real dataset is AmazonQAC:

- Source: <https://huggingface.co/datasets/amazon/AmazonQAC>
- License: CDLA-Permissive-2.0
- Relevant fields: `final\_search\_term`, `popularity`, `search\_time`, `prefixes`

The system does not train a machine learning model. It imports real search queries, stores normalized query records, indexes them for prefix lookup, and ranks suggestions using historical and recent counts.

For grading, the Docker profile was run with 100,000 unique AmazonQAC queries. The repository does not commit the full dataset because the raw AmazonQAC data is large, but the source, expected export format, and loading command are documented here.

The importer accepts the assignment's simple format:

```
query,count  
laptop,23222  
iphone 15,45222
```

It also accepts AmazonQAC CSV/JSONL fields:

```
final_search_term,popularity,search_time,prefixes
```

Place an AmazonQAC export under `data/`, for example:

```
data/amazonqac_100k.csv
```

Production-like Docker run with PostgreSQL/OpenSearch/Redis/Kafka:

```
TYPEAHEAD_DATASET_IMPORT_PATH=/data/amazonqac_100k.csv \  
TYPEAHEAD_DATASET_IMPORT_LIMIT=100000 \  
docker compose up --build
```

Optional quick run without external services:

```
mvn -Dmaven.repo.local=.m2/repository spring-boot:run
```

The quick run uses a tiny built-in seed only to let an evaluator inspect the UI and APIs immediately. The measured submission run uses the 100,000-query Docker command above.

3. API Documentation

Swagger/OpenAPI is available at `/docs` when the app is running.

API	Purpose
<code>`GET /suggest? q=iph&rank=count`</code>	Top 10 prefix suggestions sorted by historical count.
<code>`GET /suggest? q=iph&rank=hybrid`</code>	Top 10 prefix suggestions using recency-aware ranking.
<code>`POST /search`</code>	Accepts a search query and queues a search event.
<code>`GET /trending?window=1h`</code>	Returns trending searches for <code>`1h`</code> , <code>`24h`</code> , or <code>`7d`</code> .
<code>`GET /cache/debug? prefix=iph`</code>	Shows normalized prefix, selected cache node, hit/miss, key, TTL, and ring settings.
<code>`GET /metrics`</code>	Shows latency, p95, cache hit rate, DB writes, write reduction, and cache-node distribution.

Example search request:

```
curl -X POST 'http://127.0.0.1:8080/search' \  
-H 'Content-Type: application/json' \  
-d '{"query":"iphone 15"}'
```

Example suggestion response:

```
{
  "query": "iph",
  "rank": "hybrid",
  "suggestions": [
    {
      "query": "iphone 15",
      "historical_count": 850000,
      "recent_count_1h": 312,
      "recent_count_24h": 1205,
      "score": 918240
    }
  ],
  "cache": {
    "hit": true,
    "node": "cache-node-2",
    "key": "suggest:v1:1:hybrid:iph"
  },
  "latency_ms": 3.7
}
```

4. Design Choices And Trade-offs

PostgreSQL is the source of truth for query records in the Docker profile. It stores normalized queries, historical counts, last search time, and trend buckets. The trade-off is that PostgreSQL is not used as the hot prefix read path; Redis and OpenSearch handle the latency-sensitive path.

OpenSearch stores the suggestion read model. It supports prefix autocomplete with weighted ranking. This keeps suggestion reads fast, but updates are eventually consistent because search events are batched before weights are refreshed.

Redis is used as a distributed prefix cache. The application includes a Java consistent hash ring with virtual nodes so each prefix cache key maps deterministically to one cache shard. This makes cache-node ownership and remapping visible through ``/cache/debug``.

Kafka is used as the durable search-event buffer in the Docker profile. Search submissions return quickly after publishing an event. Batch workers aggregate repeated searches every 2 seconds or 500 events, reducing database and index writes. The trade-off is brief staleness: PostgreSQL remains durable, while Redis and OpenSearch may lag until the next flush or cache invalidation.

Trending ranking combines historical and recent activity:

```
score =  
  log10(historical_count + 1) * 100000  
  + log10(recent_count_1h + 1) * 60000  
  + log10(recent_count_24h + 1) * 25000
```

Recent searches are stored in 5-minute buckets. Old spikes stop dominating because they fall out of the active time window. Hybrid cache entries also use a shorter TTL than historical ranking.

5. Performance Report

Benchmark command:

```
./scripts/benchmark.sh
```

Measured full-dataset Docker results:

Metric	Value
Dataset source	AmazonQAC
Loaded PostgreSQL rows	100,000
Loaded OpenSearch documents	100,000
Suggest requests	200
Cache hits	192
Cache misses	8
Cache hit rate	96.00%
Average suggest latency	6.40 ms
P95 suggest latency	6.43 ms
Search events received	500
Kafka events published	500
DB write operations	8

Writes saved by batching	492
Write reduction	98.40%
Cache node distribution	cache-node-1: 75, cache-node-2: 100, cache-node-3: 25

The full run was verified with ``query_catalog = 100000`` in PostgreSQL and ``query_suggestions_v1 = 100000`` in OpenSearch before the benchmark was recorded.

Demo Evidence

The UI supports a search input, debounced suggestions, keyboard navigation, search submission, ranking toggle, trending panel, cache debug panel, metrics panel, loading states, and error states.

Screenshots are included in:

- ``docs/screenshots/search-ui-overall-mode.png``
- ``docs/screenshots/search-ui-suggestions.png``